

```
!pip install tensorflow matplotlib scikit-image opencv-python
```

```
import tensorflow as tf
from tensorflow.keras import layers, models
import numpy as np
import matplotlib.pyplot as plt
import cv2, os
```

```
from skimage.metrics import peak_signal_noise_ratio as psnr
from skimage.metrics import structural_similarity as ssim
```

```
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-packages (2.20.0)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: scikit-image in /usr/local/lib/python3.12/dist-packages (0.25.2)
Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.13.0.92)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.12.19)
Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: google_pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt_einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from tensorflow) (26.1)
Requirement already satisfied: protobuf>=5.28.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (5.29.6)
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.32.4)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from tensorflow) (75.2.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.3.0)
Requirement already satisfied: typing_extensions>=3.6.6 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (4.15.0)
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.1.2)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.80.0)
Requirement already satisfied: tensorboard~=2.20.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.20.0)
Requirement already satisfied: keras>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.13.2)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.0.2)
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.16.0)
Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.5.4)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: scipy>=1.11.4 in /usr/local/lib/python3.12/dist-packages (from scikit-image) (1.16.3)
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.12/dist-packages (from scikit-image) (3.6.1)
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/python3.12/dist-packages (from scikit-image) (2.37.3)
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.12/dist-packages (from scikit-image) (2026.4.11)
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.12/dist-packages (from scikit-image) (0.5)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0->tensorflow) (0.44.0)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow) (0.1.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow) (0.19.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (2025.11.11)
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.20.0->tensorflow) (3.7.0)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.20.0->tensorflow) (0.7.0)
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.20.0->tensorflow) (3.1.0)
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from werkzeug>=1.0.1->tensorboard~=2.20.0->tensorflow) (3.0.2)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.10.0->tensorflow) (3.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.10.0->tensorflow) (2.19.0)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich->keras>=3.10.0->tensorflow) (0.1.2)
```

```
from google.colab import files
uploaded = files.upload()
```

```
import zipfile
```

```
zip_path = list(uploaded.keys())[0]
```

```
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall('/content/mri_data')
```

```
print("Dataset Extracted!")
```

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable

```
def load_images(path, size=(64,64)):
    images = []
    for root, _, files in os.walk(path):
        for file in files:
            if file.endswith(('.png', '.jpg', '.jpeg')):
                img = cv2.imread(os.path.join(root,file), cv2.IMREAD_GRAYSCALE)
                img = cv2.resize(img, size)
                img = img / 255.0
                images.append(img)

    images = np.array(images,dtype=np.float32)
    return images[..., None]

x_data = load_images('/content/mri_data')
print("Total images:", x_data.shape)
```

Total images: (358, 64, 64, 1)

```
split = int(0.8 * len(x_data))
x_train, x_test = x_data[:split], x_data[split:]

def add_noise(images):
    noise = np.random.normal(0, 0.05, images.shape).astype(np.float32)
    noisy = images + noise
    return np.clip(noisy, 0., 1.).astype(np.float32)

x_train_noisy = add_noise(x_train)
x_test_noisy = add_noise(x_test)
```

```
def build_generator():
    inputs = layers.Input(shape=(64,64,1))

    # Encoder
    c1 = layers.Conv2D(64,3,padding='same',activation='relu')(inputs)
    p1 = layers.MaxPooling2D()(c1)

    c2 = layers.Conv2D(128,3,padding='same',activation='relu')(p1)
    p2 = layers.MaxPooling2D()(c2)

    # Bottleneck
    b = layers.Conv2D(256,3,padding='same',activation='relu')(p2)

    # Decoder
    u1 = layers.UpSampling2D()(b)
    u1 = layers.Concatenate()([u1, c2])
    c3 = layers.Conv2D(128,3,padding='same',activation='relu')(u1)

    u2 = layers.UpSampling2D()(c3)
    u2 = layers.Concatenate()([u2, c1])
    c4 = layers.Conv2D(64,3,padding='same',activation='relu')(u2)

    # 🌀 Quantum-Inspired Feature Modulation
    q = layers.GlobalAveragePooling2D()(c4)
    q = layers.Dense(32, activation='relu')(q)
    q = layers.Dense(64, activation='sigmoid')(q)
    q = layers.Reshape((1,1,64))(q)
    q = layers.UpSampling2D(size=(64,64))(q)

    c4 = layers.Multiply()([c4, q])

    # Output
    outputs = layers.Conv2D(1,3,padding='same',activation='sigmoid')(c4)

    return tf.keras.Model(inputs, outputs)

generator = build_generator()
generator.summary()
```

Model: "functional_2"

Layer (type)	Output Shape	Param #	Connected to
input_layer_2 (InputLayer)	(None, 64, 64, 1)	0	-
conv2d_7 (Conv2D)	(None, 64, 64, 64)	640	input_layer_2[0]...
max_pooling2d_1 (MaxPooling2D)	(None, 32, 32, 64)	0	conv2d_7[0][0]
conv2d_8 (Conv2D)	(None, 32, 32, 128)	73,856	max_pooling2d_1[...
max_pooling2d_2 (MaxPooling2D)	(None, 16, 16, 128)	0	conv2d_8[0][0]
conv2d_9 (Conv2D)	(None, 16, 16, 256)	295,168	max_pooling2d_2[...
up_sampling2d_2 (UpSampling2D)	(None, 32, 32, 256)	0	conv2d_9[0][0]
concatenate (Concatenate)	(None, 32, 32, 384)	0	up_sampling2d_2[...] conv2d_8[0][0]
conv2d_10 (Conv2D)	(None, 32, 32, 128)	442,496	concatenate[0][0]
up_sampling2d_3 (UpSampling2D)	(None, 64, 64, 128)	0	conv2d_10[0][0]
concatenate_1 (Concatenate)	(None, 64, 64, 192)	0	up_sampling2d_3[...] conv2d_7[0][0]
conv2d_11 (Conv2D)	(None, 64, 64, 64)	110,656	concatenate_1[0]...
global_average_poo... (GlobalAveragePool...)	(None, 64)	0	conv2d_11[0][0]
dense_3 (Dense)	(None, 32)	2,080	global_average_p...
dense_4 (Dense)	(None, 64)	2,112	dense_3[0][0]
reshape_1 (Reshape)	(None, 1, 1, 64)	0	dense_4[0][0]
up_sampling2d_4 (UpSampling2D)	(None, 64, 64, 64)	0	reshape_1[0][0]
multiply_1 (Multiply)	(None, 64, 64, 64)	0	conv2d_11[0][0], up_sampling2d_4[...]
conv2d_12 (Conv2D)	(None, 64, 64, 1)	577	multiply_1[0][0]

```
def build_discriminator():
    model = models.Sequential([
        layers.Conv2D(32, 3, strides=2, padding='same', input_shape=(64,64,1)),
        layers.LeakyReLU(),

        layers.Conv2D(64, 3, strides=2, padding='same'),
        layers.LeakyReLU(),

        layers.Flatten(),
        layers.Dense(1, activation='sigmoid')
    ])
    return model

discriminator = build_discriminator()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
cross_entropy = tf.keras.losses.BinaryCrossentropy()

def d_loss(real, fake):
    return cross_entropy(tf.ones_like(real)*0.9, real) + \
        cross_entropy(tf.zeros_like(fake), fake)
```

```
def g_loss(fake, generated, real):
    gan_loss = cross_entropy(tf.ones_like(fake), fake)
    l1_loss = tf.reduce_mean(tf.abs(real - generated))
    ssim_loss = 1 - tf.reduce_mean(tf.image.ssim(real, generated, max_val=1.0))

    return gan_loss + 50*l1_loss + 10*ssim_loss
```

```
g_opt = tf.keras.optimizers.Adam(0.0002)
d_opt = tf.keras.optimizers.Adam(0.0002)
```

```
def train_step(noisy, real):

    noisy = tf.cast(noisy, tf.float32)
    real = tf.cast(real, tf.float32)

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:

        generated = generator(noisy, training=True)

        real_out = discriminator(real, training=True)
        fake_out = discriminator(generated, training=True)

        gen_loss = g_loss(fake_out, generated, real)
        disc_loss = d_loss(real_out, fake_out)

    g_grad = gen_tape.gradient(gen_loss, generator.trainable_variables)
    d_grad = disc_tape.gradient(disc_loss, discriminator.trainable_variables)

    g_opt.apply_gradients(zip(g_grad, generator.trainable_variables))
    d_opt.apply_gradients(zip(d_grad, discriminator.trainable_variables))

    return gen_loss, disc_loss
```

```
EPOCHS = 50
BATCH_SIZE = 16

dataset = tf.data.Dataset.from_tensor_slices((x_train_noisy, x_train))
dataset = dataset.shuffle(1000).batch(BATCH_SIZE)
for epoch in range(EPOCHS):
    print(f"\nEpoch {epoch+1}")

    g_total, d_total, steps = 0, 0, 0

    for noisy_batch, real_batch in dataset:
        g_loss_val, d_loss_val = train_step(noisy_batch, real_batch)

        g_total += g_loss_val.numpy()
        d_total += d_loss_val.numpy()
        steps += 1

    print(f"G Loss: {g_total/steps:.4f}, D Loss: {d_total/steps:.4f}")
```

```
Epoch 1
G Loss: 23.0622, D Loss: 1.2231

Epoch 2
G Loss: 17.0980, D Loss: 1.1790

Epoch 3
G Loss: 11.4462, D Loss: 1.3499

Epoch 4
G Loss: 9.1435, D Loss: 1.3166

Epoch 5
G Loss: 7.6593, D Loss: 1.2908

Epoch 6
G Loss: 6.6288, D Loss: 1.2561

Epoch 7
G Loss: 6.0367, D Loss: 1.2125

Epoch 8
G Loss: 5.4249, D Loss: 1.1615
```

```

Epoch 9
G Loss: 5.1624, D Loss: 1.1282

Epoch 10
G Loss: 4.9154, D Loss: 1.1417

Epoch 11
G Loss: 4.6884, D Loss: 1.1586

Epoch 12
G Loss: 4.5025, D Loss: 1.2069

Epoch 13
G Loss: 4.3889, D Loss: 1.2579

Epoch 14
G Loss: 4.2218, D Loss: 1.2844

Epoch 15
G Loss: 4.1638, D Loss: 1.2960

Epoch 16
G Loss: 4.2013, D Loss: 1.3005

Epoch 17
G Loss: 4.0760, D Loss: 1.2954

Epoch 18
G Loss: 3.9629, D Loss: 1.2954

Epoch 19
G Loss: 3.9480, D Loss: 1.3023

```

```

test_img = x_test_noisy[0:1]
denoised = generator.predict(test_img)

plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
plt.title("Noisy")
plt.imshow(test_img[0].squeeze(), cmap='gray')

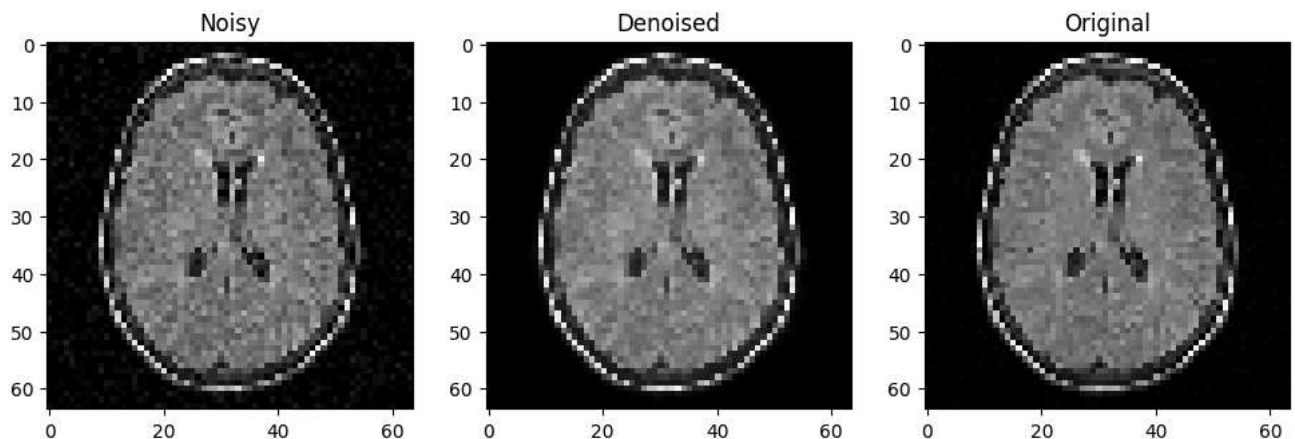
plt.subplot(1,3,2)
plt.title("Denoised")
plt.imshow(denoised[0].squeeze(), cmap='gray')

plt.subplot(1,3,3)
plt.title("Original")
plt.imshow(x_test[0].squeeze(), cmap='gray')

plt.show()

```

1/1 ————— 1s 1s/step



```

psnr_val = psnr(x_test[0], denoised[0], data_range=1)
ssim_val = ssim(x_test[0].squeeze(), denoised[0].squeeze(), data_range=1)

print("PSNR:", psnr_val)
print("SSIM:", ssim_val)

```

```
PSNR: 28.742591521961987  
SSIM: 0.8982748248914476
```